

# Aufgaben des Linux-Kernels

Wie das Betriebssystem Ressourcen, Geräte und Programme zusammenführt

1.03

# Der Kernel vermittelt zwischen Software und Hardware

## Anwendungen

Shell · Browser · Dienste · Embedded-Anwendung

## System Calls

open() · read() · write() · mmap() · socket()

## Linux-Kernel

Scheduling · Speicher · VFS · Treiber · Netzwerk

## Hardware

CPU · RAM · SSD · Netzwerk · Sensoren

# Acht Aufgaben tragen das gesamte System

---

## 01 Prozesse planen

CPU-Zeit und Ausführungsreihenfolge steuern.

## 02 Speicher verwalten

Virtuelle Adressen, Pages und Schutzbereiche organisieren.

## 03 Dateizugriffe vereinheitlichen

Dateisysteme und I/O über gemeinsame Schnittstellen ansprechen.

## 04 Geräte anbinden

Treiber, Busse und Geräte-Events koordinieren.

## 05 Interrupts verarbeiten

Asynchrone Hardware-Events annehmen und verteilen.

## 06 Netzwerk bereitstellen

Pakete, Protokolle, Routing und Sockets verarbeiten.

## 07 Zugriffe absichern

Privilegien, Berechtigungen und Sicherheitsregeln prüfen.

## 08 Ressourcen begrenzen

Namespaces und cgroups für Isolation und Limits nutzen.

# Der Scheduler verteilt verfügbare CPU-Zeit

## 01 Tasks auswählen

Der Kernel entscheidet, welcher runnable Task als Nächstes läuft.

## 02 Prioritäten berücksichtigen

Scheduling-Klassen, Prioritäten und Affinität beeinflussen die Auswahl.

## 03 Zwischen Tasks wechseln

Beim Context Switch wechselt der Ausführungskontext auf der CPU.

### TERMINAL

```
$ ps -eo pid,ni,stat,comm  
$ cat /proc/schedstat  
$ taskset -pc <PID>
```

Mehr Programme als CPU-Kerne sind möglich, weil der Kernel Ausführung zeitlich organisiert.

# Virtueller Speicher trennt und schützt Prozesse

## 01 Adressräume abbilden

Page Tables verbinden virtuelle Adressen mit physischen Pages.

## 02 Pages bedarfsgerecht verwalten

Page Faults, Page Cache und Reclaim steuern die tatsächliche Nutzung.

## 03 Zugriff schützen

Lese-, Schreib- und Ausführungsrechte begrenzen Speicherzugriffe.

### SPEICHER BEOBACHTEN

```
$ cat /proc/meminfo  
$ cat /proc/<PID>/maps  
$ cat /proc/<PID>/status
```

Jeder Prozess erhält seine eigene Sicht auf Speicher; der Kernel verwaltet die gemeinsame Hardware.

# Das VFS vereinheitlicht den Zugriff auf Dateien

## Anwendung

`open()` · `read()` · `write()`

### Virtual File System

Pfadauflösung · Inodes · File Descriptors · Rechte

**ext4**

**XFS**

**tmpfs**

**procfs / sysfs**

Einheitliche Dateioperationen funktionieren über verschiedene Dateisysteme hinweg.

# Gerätetreiber kapseln hardwareabhängiges Verhalten

## 01 Geräte erkennen

Der Kernel verbindet Geräte über Bus- und Geräteinformationen mit Treibern.

## 02 Operationen bereitstellen

Treiber implementieren die Kommunikation mit Registern, DMA und Geräten.

## 03 Lebenszyklus verwalten

Initialisierung, Laufzeit, Fehlerbehandlung und Power Management greifen zusammen.

### TREIBER UND GERÄTE

```
$ lsmod  
$ modinfo <modul>  
$ ls /sys/bus  
$ readlink -f /sys/class/net/*
```

Treiber verbergen Hardwaredetails hinter einheitlichen Geräte- und I/O-Schnittstellen.

# Interrupts übertragen Events aus der Hardware

---

## 01 Ein Event entsteht

Ein Gerät meldet etwa Netzwerkverkehr, abgeschlossene I/O oder einen Timer.

## 02 Der Kernel ordnet die IRQ zu

Die Interrupt-Infrastruktur verbindet die Quelle mit dem passenden Handler.

## 03 Die Verarbeitung wird fortgesetzt

Treiber und Kernel-Subsysteme übernehmen die weitere Bearbeitung.

### BEOBACHTEN

```
$ cat /proc/interrupts  
$ watch -n 0.5 'grep timer /proc/interrupts'
```

# Der Netzwerk-Stack verarbeitet Pakete und Protokolle

## 01 Sockets anbieten

Anwendungen senden und empfangen Daten über standardisierte Endpunkte.

## 02 Protokolle verarbeiten

Transport- und Netzwerkprotokolle kapseln, prüfen und verteilen Daten.

## 03 Geräte und Routing verbinden

Netzwerkschnittstellen, Routen und Filter bestimmen den Paketweg.

### NETZWERK BEOBACHTEN

```
$ ip address  
$ ip route  
$ ss -tuln  
$ cat /proc/net/dev
```

Eine Anwendung spricht mit einem Socket; der Kernel übernimmt Protokolle, Routing und Gerätetreiber.

# System Calls bilden den kontrollierten Übergang

## USER SPACE

**Ein Programm fordert eine geschützte Operation an.**

`open()` · `read()` · `mmap()` · `socket()`

## KERNEL SPACE

**Der Kernel prüft Parameter, Berechtigungen und Kontext.**

Ergebnis oder Fehlercode zurückgeben

Ein System Call ist kein direkter Hardwarezugriff, sondern eine kontrollierte Anfrage an den Kernel.

# Sicherheitsprüfungen begrenzen sensible Zugriffe

## 01 Identität und Rechte

UID, GID, Dateirechte und Prozess-Credentials bestimmen den Zugriff.

## 02 Capabilities

Privilegierte Operationen lassen sich in einzelne Berechtigungen aufteilen.

## 03 Linux Security Modules

Zusätzliche Richtlinien ergänzen die klassischen Zugriffskontrollen.

## 04 Seccomp

Prozesse können die Menge erlaubter System Calls einschränken.

Sicherheit entsteht im Kernel durch geprüfte Übergänge, begrenzte Privilegien und isolierte Ressourcen.

# Namespaces isolieren, cgroups begrenzen Ressourcen

---

## Namespaces

### Was darf ein Prozess sehen?

Eigene Sichten auf Prozesse, Netzwerke, Mounts, Hostnamen oder Benutzer.

## cgroups

### Wie viele Ressourcen darf er nutzen?

CPU, Speicher und weitere Ressourcen gruppieren, messen und begrenzen.

Container kombinieren Isolation der Sicht mit kontrollierter Ressourcennutzung.

# Ein Dateizugriff aktiviert mehrere Subsysteme

---

## 01 read() im Userspace

Die Anwendung fordert Daten über einen File Descriptor an.

## 02 System Call und Sicherheitsprüfung

Der Kernel übernimmt den kontrollierten Übergang und prüft Zugriffsrechte.

## 03 VFS und Dateisystem

Pfad, Inode, File-Objekt und Page Cache führen zum passenden Dateisystem.

## 04 Treiber, I/O und Rückgabe

Bei Bedarf greift ein Gerätetreiber auf das Medium zu; das Ergebnis geht zurück.

Subsysteme sind fachlich getrennt, arbeiten bei realen Operationen jedoch eng zusammen.

# Kernel, Distribution und Anwendung sind nicht dasselbe

---

## 01 Linux-Kernel

Verwaltet Hardware, Prozesse, Speicher und privilegierte Systemfunktionen.

## 02 Distribution

Bündelt Kernel, Paketverwaltung, Bibliotheken, Dienste und Systemwerkzeuge.

## 03 Anwendung

Läuft im Userspace und nutzt Bibliotheken sowie System Calls.

Ubuntu, Debian oder Fedora sind Distributionen; Linux bezeichnet den Kernel.

# Deine Systemanalyse macht Kernaufgaben sichtbar

---

**Scheduling**`ps · /proc/schedstat`

---

**Speicher**`/proc/meminfo · /proc/<PID>/maps`

---

**Dateisysteme**`mount · /proc/filesystems`

---

**Geräte und Treiber**`lsmod · /sys/devices · /sys/bus`

---

**Interrupts**`/proc/interrupts`

---

**Netzwerk**`ip address · /proc/net/dev`

---